

LAPORAN PENGEMBANGAN SISTEM MANAJEMEN
TUGAS KKN



Oleh:

M. Khalid Al Rejeki
F1D02310122
Kelas D

1. Deskripsi Aplikasi dan Tujuan Project

a. Deskripsi Aplikasi

Sistem Manajemen Tugas KKN adalah sebuah aplikasi desktop berbasis antarmuka grafis (GUI) yang dirancang secara khusus untuk mengelola, mencatat, dan memantau rincian kegiatan selama program Kuliah Kerja Nyata (KKN). Aplikasi ini dikembangkan menggunakan bahasa pemrograman Python terintegrasi dengan *framework* PySide6 (Qt for Python), serta menggunakan basis data lokal SQLite.

b. Tujuan Project

- Digitalisasi Pencatatan: Menggantikan pencatatan program kerja (proker) manual menjadi sistem digital yang rapi dan terpusat.
- Efisiensi Manajemen Waktu: Mempermudah tim KKN dalam melacak status kegiatan (Belum Mulai, Sedang Berjalan, Selesai) secara *real-time*.
- Kemandirian Sistem: Menyediakan perangkat lunak yang dapat berjalan sepenuhnya secara luring (*offline*) di berbagai kondisi lapangan KKN yang mungkin minim akses internet.

2. Struktur Project dan Penerapan *Separation of Concerns* (SoC)

Aplikasi ini tidak ditulis dalam satu *file* tunggal yang menumpuk, melainkan mengadopsi prinsip desain perangkat lunak *Separation of Concerns* (SoC). Prinsip ini memisahkan program ke dalam modul-modul fungsional yang independen agar lebih mudah dikelola, dipelihara, dan diperbarui.

Berikut adalah struktur direktori proyek beserta peran masing-masing komponen:

a. main.py (Entry Point):

Bertugas sebagai file pemicu jalannya aplikasi. Modul ini menginisialisasi `QApplication`, memuat *style sheet* eksternal, dan menampilkan antarmuka utama.

b. database.py (Data/Model Layer):

Modul yang mengisolasi seluruh logika yang berkaitan dengan interaksi basis data. Tidak ada kode antarmuka di file ini; fungsinya murni mengeksekusi *query* SQLite (Create, Read, Update, Delete).

c. ui_main.py (Presentation & Controller Layer):

Menangani tata letak jendela utama, *Menu Bar*, tabel (`QTableWidget`), serta berisi logika *Signals & Slots* yang menghubungkan aksi pengguna dengan manipulasi data di basis data.

d. ui_dialog.py (Presentation Layer):

Berisi definisi kelas `QDialog` untuk form input kegiatan. Dipisahkan dari jendela utama agar desain antarmuka form lebih fleksibel dan *reusable*.

e. style.qss (Styling Layer):

File eksternal yang memuat sintaks QSS (Qt Style Sheets). Berfungsi untuk memisahkan desain visual (warna, *padding*, *border*) dari logika inti program.

3. Struktur Project dan Penerapan *Separation of Concerns* (SoC)

Aplikasi menggunakan **SQLite** karena sifatnya yang *serverless* dan terintegrasi langsung dengan Python, sehingga sangat cocok untuk skala aplikasi desktop lokal. Basis data (kkn_tasks.db) terdiri dari satu tabel utama bernama tasks dengan skema (*schema*) sebagai berikut:

Nama Kolom	Tipe Data	Keterangan
id	Integer	<i>Primary Key</i> , <i>Auto Increment</i> (Penanda unik).
nama_kegiatan	text	Menyimpan nama atau judul program kerja
lokasi	text	Menyimpan detail tempat pelaksanaan kegiatan.
tanggal	text	Disimpan dalam format yyyy-MM-dd.
penanggung_jawab	text	Nama anggota KKN yang memimpin kegiatan.
status	text	Menyimpan status (<i>dropdown</i>): Belum Mulai / Berjalan / Selesai.
deskripsi	text	Catatan detail mengenai teknis kegiatan.

4. Struktur Project dan Penerapan *Separation of Concerns* (SoC)

a. Tambah Kegiatan (Create)

Saat pengguna mengklik tombol "Tambah Kegiatan", aplikasi memicu *signal* untuk memunculkan QDialog dari modul ui_dialog.py. Form input menggunakan QFormLayout. Ketika tombol "Simpan" ditekan, fungsi *slot* akan mengekstrak data dari input *field*, memvalidasinya (memastikan nama kegiatan tidak kosong), dan mengirimkannya ke metode insert_task() di modul database.py.

b. Tampilkan Kegiatan (Read)

Setiap kali aplikasi pertama kali dijalankan atau setelah terjadi perubahan data, metode load_data() akan dipanggil. Metode ini akan mengambil seluruh data dari basis data menggunakan metode get_all_tasks(), menghapus baris tabel yang ada di layar, dan menggunakan perulangan *for* dengan indeks untuk menyusun objek QTableWidgetItem ke dalam baris dan kolom yang tepat.

c. Edit Kegiatan (Update)

Pengguna dapat memilih salah satu baris pada tabel, lalu menekan tombol "Edit". Aplikasi akan mengambil *ID Task* yang tersembunyi/terlihat di kolom pertama, lalu memunculkan form dialog yang sudah diisi otomatis (*autofill*) menggunakan fungsi set_data(). Setelah diperbarui, data dikirim ke update_task() untuk mengubah baris terkait di basis data.

d. Hapus Kegiatan (Delete)

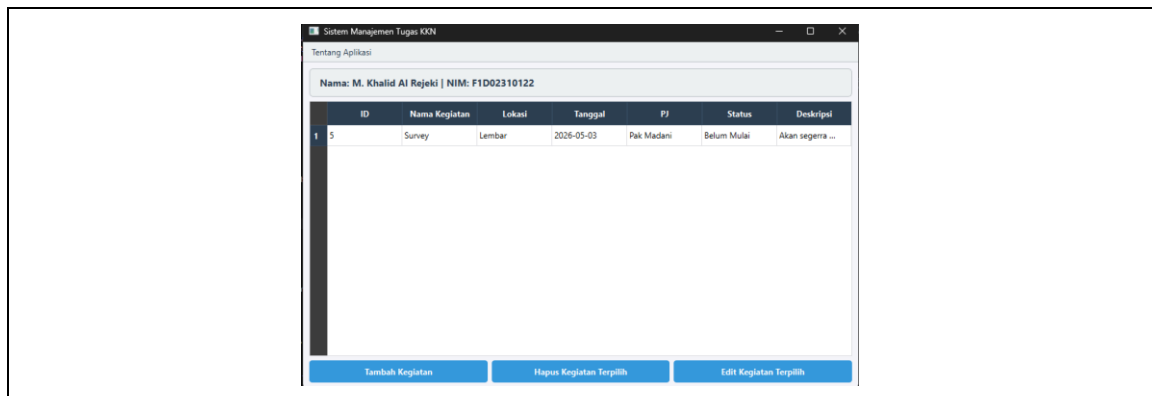
Fitur ini dilengkapi dengan mekanisme pencegahan *human error*. Saat tombol hapus ditekan, aplikasi akan terlebih dahulu memunculkan dialog peringatan kelas QMessageBox. Jika pengguna mengonfirmasi (klik 'Yes'), *ID Task* akan diteruskan ke fungsi eksekutor DELETE di SQLite, dan tabel UI akan di-*refresh*.

e. Menu Bar & Informasi Identitas

Terdapat komponen antarmuka yang bersifat statis (*read-only*). Informasi pengembang ditampilkan di bagian antarmuka utama melalui QLabel dan juga disematkan di dalam QMenuBar. Saat tombol "Tentang Aplikasi" diklik, ia mengeksekusi pemanggilan dialog informasi interaktif.

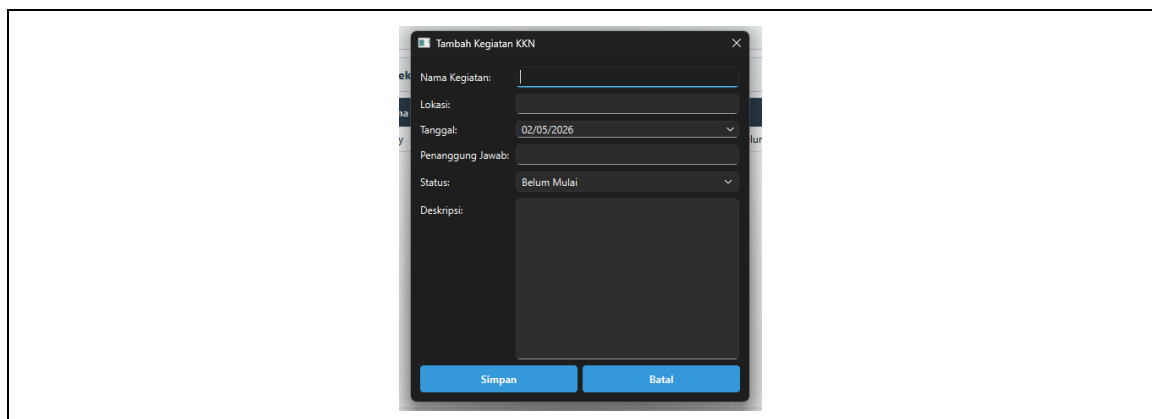
5. Screenshot UI

a. UI Dashboard Sistem



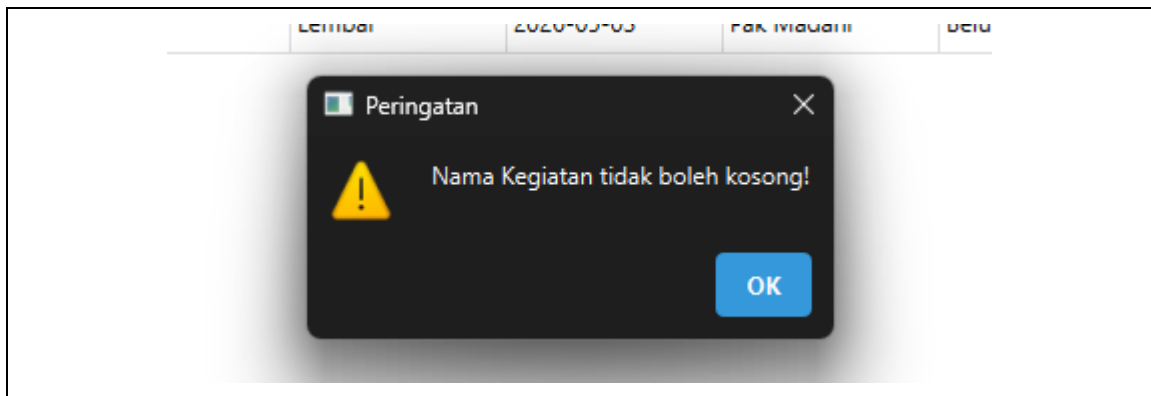
Tampilan awal sistem dibagi menjadi 3 area utama. Bagian paling atas memuat menubar walau cuman satu fitur saja yaitu tentang aplikasi bagian tengah menggunakan QTableWidgetItem untuk memuat data dari database dan bagian bawah memiliki 3 tombol aksi.

b. UI Form Input



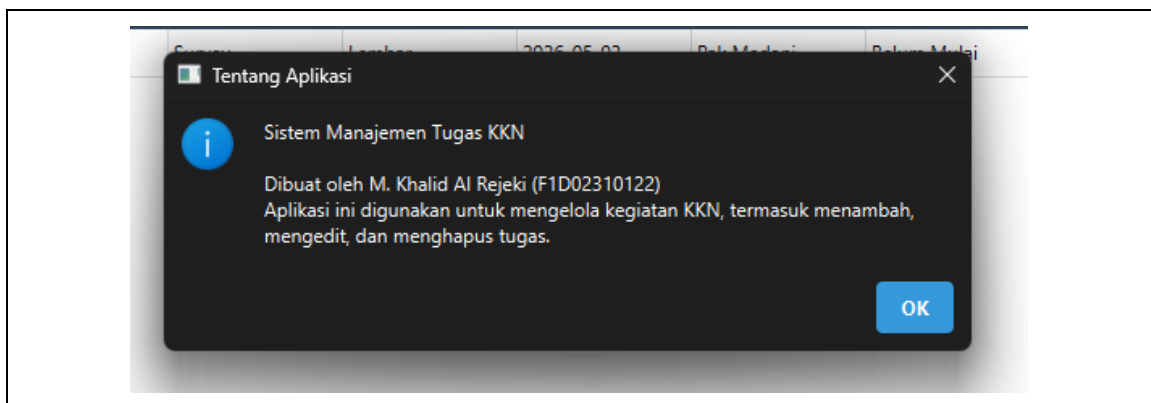
Tampilan dialog tambah kegiatan dibagi menjadi 2 area utama. Bagian atas memuat form isian data yang terdiri dari 6 *input field* menggunakan kombinasi komponen seperti QLineEdit, QDateEdit, QComboBox, dan QTextEdit yang disusun rapi menggunakan QFormLayout, dan bagian bawah memiliki 2 tombol aksi yaitu Simpan dan Batal.

c. UI Pop Up MessageBox Warning



Tampilan ini menunjukkan implementasi kotak dialog validasi menggunakan komponen QMessageBox yang berfungsi sebagai fitur pencegahan kesalahan masukan pengguna. Dialog berlatar belakang gelap ini memuat ikon peringatan standar berupa segitiga kuning dan pesan teks "Nama Kegiatan tidak boleh kosong!", yang dirancang untuk muncul ketika pengguna mencoba menyimpan data tanpa mengisi *field* utama yang diwajibkan. Pada bagian bawah kanan dialog, terdapat satu tombol aksi tunggal bertuliskan "OK" dengan *styling* biru yang berfungsi untuk menutup peringatan dan mengembalikan pengguna ke form pengisian data.

d. UI Pop Up MessageBox Tentang Aplikasi



Tampilan ini menunjukkan implementasi kotak dialog informasi menggunakan komponen QMessageBox yang berfungsi untuk menampilkan menu "Tentang Aplikasi". Dialog berlatar belakang gelap ini memuat ikon informasi standar berupa lingkaran biru dengan huruf 'i' di sisi kiri, disandingkan dengan teks yang memuat nama aplikasi "Sistem Manajemen Tugas KKN", identitas pembuat yaitu M. Khalid Al Rejeki (F1D02310122), serta deskripsi singkat mengenai fungsionalitas utama aplikasi. Pada bagian sudut kanan bawah, terdapat satu tombol aksi tunggal bertuliskan "OK" dengan *styling* biru yang berfungsi untuk menutup jendela informasi tersebut.

6. Link Link

Link Youtube : <https://youtu.be/gQAuiJyS-sk>

Link GitHub : <https://github.com/LepangMbojo/pv26-miniproject-LogbookKKN-F1D0231022>